

UNIVERSIDAD TÉCNICA PARTICULAR DE LOJA

Ingeniería en Ciencias de la Computación
Ingeniería de Requerimientos — Junio 2026

INFORME DE ROL BACKEND

SAMR — Sistema de Atención Médica Remota

Transformado mediante IA e Interfaces Conversacionales

Antonella Parra Carrión

Desarrolladora Backend — Equipo LLdE

1. Introducción

Este informe documenta mi participación como Desarrolladora Backend en el proyecto SAMR (Sistema de Atención Médica Remota), desarrollado en la asignatura de Ingeniería de Requerimientos de la UTPL. El sistema SAMR busca brindar atención médica remota de alta calidad mediante cinco módulos funcionales integrados: Gestión de Usuarios y Acceso, Triage Inteligente, Monitoreo Biométrico Predictivo, Teleconsulta Médica y Seguridad/Auditoría. Lo que hace especial a este sistema es que utiliza Med-Gemini, el modelo de IA multimodal de Google entrenado específicamente con datos clínicos, como motor de razonamiento médico central.

Mi rol estuvo enfocado en analizar cómo los requerimientos funcionales y no funcionales definidos en el documento de visión (SAMR_Vision_v3_Final) impactaban directamente en el diseño de la capa backend. Esto significó definir flujos de información, lógica de negocio, mecanismos de integración con APIs externas, y las responsabilidades de cada servicio que el backend debería exponer.

Una de las cosas que más me llamó la atención durante el proyecto fue entender que el backend no es solo "donde se guarda la información". Es el cerebro que conecta todo: la app del paciente, los dispositivos IoT, el motor de IA, el historial clínico, las APIs del IESS y el MSP, y el módulo de auditoría. Sin un backend bien pensado desde la etapa de requerimientos, ninguno de esos componentes puede colaborar de forma coherente.

Cabe aclarar que el proyecto se centró en la etapa de análisis y diseño de requerimientos, no en la implementación. Por eso, las actividades descritas aquí corresponden a propuestas de diseño, decisiones de arquitectura y especificación de servicios, fundamentadas en los casos de uso y los 22 requerimientos funcionales definidos por el equipo.

2. Contexto del Sistema y Arquitectura General

El sistema SAMR v3.3 es el resultado de múltiples iteraciones del equipo LLdE. Empezamos con 10 módulos, luego los consolidamos a 5 esenciales, integramos Med-Gemini en la versión 3.2, y finalmente en la v3.3 separamos correctamente los módulos M2 y M3 para distinguir entre el triaje iniciado por síntomas del paciente y las alertas automáticas generadas por los dispositivos IoT. Cada versión me ayudó a entender mejor cómo los requerimientos evolucionan y cómo eso impacta en las decisiones del backend.

La arquitectura que propuse para el backend sigue un enfoque modular orientado a servicios, donde cada módulo funcional del sistema tiene su propio conjunto de servicios, pero todos comparten las capas transversales de seguridad y auditoría. Esta decisión responde directamente a los requerimientos de escalabilidad (RNF-02: 5.000 usuarios concurrentes) y disponibilidad (RNF-11: $\geq 99.5\%$ uptime).

2.1 Capas de la Arquitectura Backend Propuesta

Capa	Responsabilidad principal
Servicios de Negocio	Lógica de cada módulo funcional: registros, triaje, monitoreo, teleconsulta, auditoría.

Persistencia	Almacenamiento de historial clínico, solicitudes, logs y datos biométricos históricos. Integridad y trazabilidad.
Integración	Comunicación con API IESS, API Red Salud Pública, Med-Gemini, sistema de recetas y servicios de emergencia bajo HL7/FHIR.
Seguridad	Autenticación MFA + OAuth 2.0, control de acceso por roles (RBAC), cifrado AES-256 en reposo y TLS 1.3 en tránsito.
Auditoría Transversal	Logs inmutables con hash criptográfico para cada operación crítica. Incluye decisiones de Med-Gemini y conversaciones con el bot.

Una decisión técnica importante fue proponer que la interoperabilidad con sistemas externos se manejara mediante el estándar HL7/FHIR. Este estándar no fue simplemente una opción tecnológica, sino que fue identificado desde temprano como un requisito de arquitectura (RF-02, R-02) porque sin él, la integración con el IESS y el MSP ecuatoriano sería prácticamente inviable.

3. Análisis por Módulo Funcional

A continuación describo el análisis que realicé para cada módulo, con énfasis en las responsabilidades backend identificadas, los requerimientos que las fundamentan, y lo que aprendí de cada uno. Los diagramas de casos de uso (archivos .drawio del proyecto) fueron la base visual de este análisis.

3.1 M1 — Gestión de Usuarios y Acceso (RF-01, RF-02, RF-18)

Este fue el módulo con el que empecé, y en apariencia parece el más sencillo: registro, login, roles. Pero cuando analicé el diagrama de casos de uso de M1 y los requerimientos, me di cuenta de que tiene tres flujos críticos que el backend debe coordinar de manera muy precisa.

El primero es el flujo de registro con MFA (RF-01). El backend no solo crea el perfil: debe asignar el rol correcto (paciente ST01, médico ST02, administrativo ST03), gestionar las credenciales y garantizar autenticación multifactor obligatoria. El RNF-01 exige 0 accesos no autorizados en auditorías, lo que significa que este flujo no puede tener ningún punto débil.

El segundo es la validación de elegibilidad con el IESS (RF-02). El backend debe consultar la API del IESS en tiempo real para verificar la cobertura del paciente antes de activar servicios. Esto introduce una dependencia externa (D-01) que el backend debe manejar con tolerancia a fallos: si la API del IESS no responde, el sistema no puede simplemente bloquearse.

El tercero, y el que más me hizo reflexionar, fue el registro del consentimiento LOPDP (RF-18). Ecuador tiene una ley de protección de datos personales que obliga a obtener consentimiento explícito del paciente antes de acceder a su historial o usar sus conversaciones con el bot para reentrenar a Med-Gemini. El backend debe registrar este consentimiento como condición previa al uso del sistema, y ese registro debe ser auditable.

3.2 M2 — Triage Inteligente y Gestión de Solicitudes (RF-03 al RF-08)

M2 fue, sin dudas, el módulo más interesante de analizar. El diagrama de casos de uso muestra 7 casos de uso diferenciados dentro del mismo módulo, y entender cómo todos colaboran me tomó varias revisiones del documento de visión.

El flujo comienza con el bot conversacional (RF-03), que guía al paciente paso a paso para crear una solicitud de reporte de síntomas. Lo que el backend hace aquí es recibir la solicitud del bot y

pasarla a Med-Gemini (RF-05) para que aplique razonamiento clínico especializado y clasifique el nivel de riesgo con XAI. Esta explicabilidad (RNF-15) no es opcional: es un requisito de arquitectura porque el MSP necesita poder auditar cada decisión del sistema.

Pero hay un segundo tipo de solicitud que fue introducido en la v3.3 y que complica un poco más el diseño del backend: las alertas de emergencia con datos IoT (diferenciadas en el diagrama como M02-C04). Cuando el monitoreo biométrico detecta una anomalía, el sistema genera una alerta directa que lleva consigo las lecturas del momento exacto de la detección. El backend debe procesar ambos tipos de solicitud de forma asíncrona (RF-06, RNF-05: en ≤ 5 segundos el 99% de los casos) para que el sistema no se sature bajo alta concurrencia.

El motor de matching inteligente (RF-07) fue uno de los aspectos que más me costó conceptualizar. El backend debe asignar cada caso al médico y centro más idóneo según especialidad, disponibilidad, ubicación y tiempo de respuesta estimado. Esto requiere que el backend mantenga un modelo de disponibilidad actualizado en tiempo real, lo cual tiene implicaciones sobre cómo se diseña la base de datos de disponibilidad y con qué frecuencia se actualiza.

3.3 M3 — Monitoreo Biométrico Predictivo (RF-09 al RF-12)

M3 fue el módulo que más me hizo entender los requerimientos no funcionales. En el papel, suena simple: recibir datos de dispositivos IoT y generar alertas. En la práctica, el RNF-12 establece que las alertas críticas deben entregarse en ≤ 30 segundos en el 99.9999% de los eventos. Ese nivel de exigencia es brutal para un sistema que está procesando flujos continuos de EKG, EEG y oxímetro simultáneamente.

El backend debe manejar la captura continua de datos biométricos (RF-09) y pasarlos a Med-Gemini (RF-10) para que detecte patrones precursores de eventos críticos. Lo que me pareció especialmente inteligente en el diseño es que el sistema no espera a que ocurra el evento: Med-Gemini detecta los patrones que preceden al evento, lo que da tiempo de reacción. El RNF-07 exige latencia máxima de 1 segundo en procesamiento de flujos IoT en el 99.9999% de las muestras.

Un aspecto que el diagrama de M3 deja claro y que el informe original no abordaba suficientemente es la separación de responsabilidades entre Med-Gemini y el bot. Med-Gemini detecta y genera la alerta con explicación clínica. El bot (RF-11) comunica esa alerta al paciente en lenguaje comprensible, no la genera. Esta distinción es importante para el diseño del backend porque define dos contratos de servicio diferentes: uno con el motor de IA y otro con la interfaz conversacional.

También identifiqué que el backend necesita distinguir entre dos tipos de almacenamiento: el flujo continuo de mediciones (almacenamiento de alta frecuencia, posiblemente en una base de datos de series temporales) y el registro histórico vinculado al historial clínico del paciente (almacenamiento relacional estructurado). Esta separación tiene implicaciones directas sobre el modelo de datos y sobre el rendimiento del sistema.

3.4 M4 — Teleconsulta y Atención Médica (RF-13, RF-14, RF-15, RF-16, RF-21)

M4 concentra la mayor complejidad lógica de todo el sistema, y los cinco requerimientos funcionales que le corresponden lo demuestran. Cuando analicé el diagrama de casos de uso de M4, me llamó mucho la atención el caso de uso RF-21: Rechazar solicitud de emergencia con

reasignación inteligente. Este caso de uso no estaba en el informe original y es uno de los más complejos del sistema.

El flujo completo de RF-21 funciona así: un médico recibe una solicitud de emergencia pero no puede atenderla (sobrecarga, falta de especialidad, indisponibilidad). Selecciona un motivo obligatorio, y el backend activa el sistema de reasignación inteligente. Si no encuentra un médico disponible en ≤ 5 minutos (RNF-18), escala automáticamente al Centro de Asistencia o al sistema de emergencias externo. El backend debe gestionar este temporizador, esta lógica de escalación y la notificación al paciente del nuevo tiempo de atención estimado, todo en paralelo.

El ciclo completo de teleconsulta (RF-13) también requiere que el backend coordine varios componentes al mismo tiempo: el canal de video/audio de baja latencia (RNF-09: ≤ 200 ms en el 95% de las sesiones), la carga del historial clínico completo integrado desde el IESS y el MSP bajo HL7/FHIR, y las sugerencias diagnósticas de Med-Gemini con XAI visibles para el médico en tiempo real. El médico puede validar o rechazar cada sugerencia, y esa decisión también queda registrada en el log de auditoría.

Los requerimientos RF-14 (recetas digitales con firma electrónica) y RF-15 (derivación de emergencia presencial) completan el ciclo. El RF-15 se conecta directamente con el sistema externo de emergencias, lo que introduce otra dependencia que el backend debe gestionar con tolerancia a fallos.

3.5 M5 — Seguridad, Auditoría y Cumplimiento (RF-16 al RF-20)

M5 no es un módulo aislado: es una capa transversal que atraviesa todos los demás módulos. El diagrama de casos de uso de M5 lo muestra claramente: el DPO (ST06), el MSP (ST05) y el equipo MLOps (ST07) interactúan directamente con este módulo para supervisar el cumplimiento, acceder a los logs y auditar la equidad de Med-Gemini.

Lo que más me impactó de este módulo fue el RNF-16: Med-Gemini no debe presentar sesgos discriminatorios verificables en sus decisiones clínicas. El equipo MLOps (ST07) debe ejecutar auditorías de equidad antes del despliegue y semestralmente, y el DPO supervisa que los datos de configuración del modelo representen adecuadamente la población ecuatoriana. Esto le da al backend una responsabilidad que va mucho más allá de la lógica técnica: es una responsabilidad ética.

Los logs de auditoría inmutables (RF-20, RNF-13) son quizás la característica más crítica del backend para el cumplimiento regulatorio. Cada log lleva un hash criptográfico que garantiza que no puede ser alterado. Incluyen decisiones de Med-Gemini, conversaciones con el bot, y todas las operaciones críticas del sistema. El RTO del sistema es ≤ 4 h con RPO ≤ 24 h (RNF-14), lo que implica backups automáticos diarios y un plan de recuperación bien definido.

4. Objetivos del Rol Backend

El trabajo en el componente backend estuvo guiado por los siguientes objetivos, todos derivados directamente del análisis de los requerimientos del proyecto:

- Analizar y proponer la lógica de negocio de cada módulo funcional, traduciendo los casos de uso en operaciones concretas sobre los datos.

- Definir la estructura de persistencia de datos médicos, garantizando integridad, trazabilidad y conformidad con la LOPDP.
- Especificar los mecanismos de comunicación con sistemas externos: API IESS, API Red de Salud Pública bajo HL7/FHIR, Med-Gemini, sistema de recetas y sistema de emergencias.
- Proponer esquemas de autenticación MFA + OAuth 2.0, control de acceso por roles (RBAC) y registro de auditoría inmutable.
- Identificar los contratos de servicio entre el backend, el motor Med-Gemini y el bot conversacional en los módulos M2, M3 y M4.
- Asegurar que todas las decisiones de diseño respetasen los requerimientos no funcionales: disponibilidad $\geq 99.5\%$, escalabilidad para 5.000 usuarios concurrentes, alertas en ≤ 30 segundos y latencia de teleconsulta $\leq 200\text{ms}$.

5. Resumen de Aportes por Caso de Uso

La siguiente tabla sintetiza los aportes principales del componente backend por cada módulo, referenciando los requerimientos funcionales y no funcionales que los fundamentan.

Módulo	CU / RF clave	Aportes Backend Principales	RNF Asociado
M1 — Gestión Usuarios	RF-01, RF-02, RF-18	Flujo de registro con MFA, validación de elegibilidad IESS, registro de consentimiento LOPDP, control de acceso por roles.	RNF-01, RNF-02
M2 — Triage	RF-03 a RF-08	Bot conversacional como primera línea, integración Med-Gemini con XAI, procesamiento asíncrono de dos tipos de solicitud, motor de matching, panel de estados en tiempo real.	RNF-05, RNF-06, RNF-15
M3 — Monitoreo	RF-09 a RF-12	Captura continua de flujos IoT, detección predictiva con Med-Gemini, separación entre almacenamiento en tiempo real e histórico, alertas multicanal $\leq 30\text{s}$.	RNF-07, RNF-08, RNF-12
M4 — Teleconsulta	RF-13 a RF-16, RF-21	Ciclo completo de consulta con historial HL7/FHIR, sugerencias XAI de Med-Gemini, receta digital con firma electrónica, derivación de emergencia, reasignación inteligente con escalación automática en ≤ 5 min.	RNF-09, RNF-15, RNF-18
M5 — Seguridad	RF-16 a RF-20	Cifrado AES-256/TLS 1.3, logs inmutables con hash criptográfico, consentimiento LOPDP, repositorio centralizado con retención de 5 años, auditoría de equidad de Med-Gemini.	RNF-13, RNF-14, RNF-15, RNF-16

6. Consideraciones de Seguridad y Protección de Datos

La naturaleza del sistema —información médica de alta sensibilidad, decisiones automatizadas de IA con impacto clínico real, integración con entidades del estado ecuatoriano— hace que la seguridad sea un eje transversal ineludible. No es un módulo adicional: es una responsabilidad que el backend debe incorporar en cada capa de su diseño.

Desde el análisis de requerimientos identifiqué las siguientes capas de protección que el backend debe garantizar:

- Autenticación MFA obligatoria + OAuth 2.0 para todos los roles (RNF-01), con 0 tolerancia a accesos no autorizados.
- Control de acceso basado en roles (RBAC): paciente, médico, administrativo, DPO y equipo MLOps tienen permisos diferenciados. El DPO tiene acceso exclusivo al módulo de auditoría y supervisión del cumplimiento LOPDP.
- Cifrado AES-256 en reposo y TLS 1.3 en tránsito para todos los datos, incluyendo las conversaciones con el bot (RF-17).
- Validación estricta de entradas en todos los endpoints para prevenir inyecciones y manipulación de datos.
- Logs de auditoría inmutables con hash criptográfico (RF-20), listos para inspección del MSP. Incluyen decisiones de Med-Gemini y conversaciones con el bot.
- Auditorías de equidad de Med-Gemini (RNF-16) antes del despliegue y semestralmente, ejecutadas por ST07 con supervisión del DPO.
- Consentimiento LOPDP registrado como precondition para el uso del historial y para el reentrenamiento de modelos con datos de conversación (RF-18, R-07).

Analizar estos requerimientos de seguridad fue especialmente valioso porque me obligó a pensar en el backend no solo como proveedor de lógica de negocio, sino como guardián de información con implicaciones legales, éticas y regulatorias concretas. En Ecuador, el incumplimiento de la LOPDP tiene consecuencias legales reales.

7. Retos Identificados y Reflexiones Técnicas

7.1 Med-Gemini como motor central: un contrato de servicio complejo

Integrar un modelo de IA como Med-Gemini no es lo mismo que llamar a una API simple. El backend necesita definir tres contratos de servicio diferentes con Med-Gemini: uno para el triaje clínico en M2 (clasificación de síntomas con XAI), otro para la detección predictiva de anomalías biométricas en M3 (análisis de flujos IoT en tiempo real), y otro para el soporte diagnóstico durante la teleconsulta en M4 (sugerencias con XAI visibles para el médico). Cada uno tiene latencias, formatos de entrada/salida y requisitos de explicabilidad distintos.

7.2 Procesamiento en tiempo real bajo restricciones extremas

El RNF-07 exige latencia máxima de 1 segundo en procesamiento de flujos IoT en el 99.9999% de las muestras, y el RNF-12 exige entrega de alertas críticas en ≤ 30 segundos en el 99.9999% de los eventos. Estas métricas son de nivel industrial. Antes de este proyecto no había tenido que pensar en términos de nueves de disponibilidad, y entender qué significa el 99.9999% de 86.400 segundos diarios (menos de 0.09 segundos de margen de error al día) fue revelador sobre la diferencia entre un sistema que funciona y un sistema que es confiable en producción.

7.3 Interoperabilidad con sistemas del estado ecuatoriano

Las APIs del IESS y la Red de Salud Pública son sistemas externos que el backend no controla. El riesgo RR-03 del documento de visión lo reconoce explícitamente: si esas APIs no están disponibles, el sistema no puede validar la elegibilidad del paciente ni importar su historial. El backend necesita implementar un modo degradado que le permita funcionar de manera limitada cuando estas dependencias fallen, y definir claramente qué funciones quedan bloqueadas y cuáles pueden continuar.

7.4 Consistencia de datos entre múltiples centros médicos

El RF-07 (motor de matching) y el RF-13 (teleconsulta con historial completo) requieren que el backend mantenga información actualizada sobre la disponibilidad y los datos clínicos de múltiples centros del consorcio médico. La sincronización de historiales bajo HL7/FHIR entre instituciones introduce problemas de consistencia que en esta etapa del proyecto identificamos como riesgo técnico. No llegamos a definir un protocolo de resolución de conflictos en detalle, pero al menos lo dejamos documentado como algo que deberá abordarse en fases posteriores.

7.5 La explicabilidad como requisito no negociable

El RNF-15 establece que el 100% de las decisiones generadas por Med-Gemini deben incluir explicación comprensible para el paciente, el médico y el auditor. Esto cambia completamente cómo el backend interactúa con el motor de IA: no es suficiente con recibir una clasificación de riesgo o una sugerencia diagnóstica. El backend también debe recibir, almacenar y transmitir la explicación de por qué Med-Gemini llegó a esa conclusión. Esta es una responsabilidad técnica, pero también ética.

8. Aprendizajes Personales y Competencias Desarrolladas

Uno de los aprendizajes más importantes durante el proyecto fue entender que muchas decisiones técnicas aparecen incluso antes de escribir una sola línea de código. Antes veía el backend principalmente como una capa encargada de bases de datos y lógica del servidor; después de este proyecto comprendí que funciona como el punto de unión entre todos los componentes del sistema.

También me llamó mucho la atención descubrir cuánto pueden afectar los requerimientos no funcionales al diseño. Al principio parecían simples números o restricciones dentro del documento, pero posteriormente entendí que terminan condicionando la arquitectura completa.

Competencias desarrolladas:

- Análisis y refinamiento de requerimientos
- Diseño de arquitectura backend orientada a servicios
- Pensamiento crítico y resolución de problemas
- Integración mediante APIs externas
- Documentación técnica y trazabilidad

9. Conclusiones

Participar como Desarrolladora Backend en el proyecto SAMR fue una experiencia que cambió cómo entiendo la ingeniería de software. Antes, "backend" era simplemente donde vivía la lógica del servidor. Después de este proyecto, entiendo que el backend es el contrato entre todos los componentes del sistema: define qué es posible, qué es seguro y qué es auditable.

El aprendizaje más importante fue confirmar que las decisiones técnicas comienzan mucho antes de escribir código. Un requerimiento como "alertas críticas en ≤ 30 segundos en el 99.9999% de los eventos" no es solo un número: es una restricción de arquitectura que descarta múltiples opciones de diseño desde el principio. Si no se identifica en la etapa de requerimientos, se descubre en producción cuando ya es costoso corregirlo.

La integración de Med-Gemini como motor clínico central fue el aspecto más desafiante de conceptualizar. Aprendí que trabajar con IA en un contexto médico no se trata solo de conectar una API: se trata de definir contratos de servicio precisos, garantizar explicabilidad, auditar equidad y asumir responsabilidad ética por las decisiones automatizadas. El RNF-16 sobre sesgos discriminatorios fue el requerimiento que más me hizo reflexionar sobre el impacto real que puede tener el software que construimos.

Finalmente, trabajar con el equipo LLdE en la evolución del documento de visión desde la v1.0 hasta la v3.3 me mostró que los requerimientos son vivos: cambian, se refinan y a veces se contradicen entre versiones. La habilidad de gestionar esa evolución sin perder la trazabilidad entre stakeholders, módulos y requerimientos es, probablemente, la competencia más valiosa que desarrollé en esta asignatura.